7. The 10-Million Burger Giveaway

> **Interviewer:** Here's a fun one. A fast-food chain wants to run a promo: give away **10 million free burgers in a 10-minute window**. A user opens the app, taps **"Claim my burger."** If burgers are still available, they get a claim code. If it's already been claimed out — or if *they* already claimed one — we show "*Your burger is on the way*". Design the backend.

Candidate: Love it. Before I design, let me make sure I understand the shape of this, because the numbers hint at the real challenge.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Is it exactly 10M burgers total, hard cap — we must **never give away 10,000,001**? 2. One burger per user, right? A user tapping twice shouldn't get two. 3. The 10-minute window — do we expect load spread evenly, or a **spike at second zero**? 4. What does "your burger is on the way" mean — is that shown both when *they already claimed* AND when *sold out*, or are those two different messages?
>
> **Interviewer:** Hard cap of exactly 10M, never over. One per user. Assume a **massive spike the instant it opens** — everyone's been told "9:00 AM sharp." "On the way" is shown if *this user already has one*; if we're sold out, show "*Sorry, all gone.*"

Candidate: Then let me write the requirements down.

Functional - `POST /claim` → returns one of: `GRANTED (code)`, `ALREADY_CLAIMED`, `SOLD_OUT`. - Exactly-once per user; global hard cap of 10M grants. - Idempotent: a user retrying a timed-out tap must not consume a second burger.

Non-functional — this is where it's interesting - **Correctness:** never exceed 10M (strong guarantee on the *counter*). - **Extreme write spike:** this is NOT a read problem. Every request is a *write attempt* against the same shared resource — a classic **thundering herd on a single hot counter**. - **Availability + graceful degradation:** when sold out, saying "all gone" fast is fine; we must not fall over. - **Latency:** a tap should resolve in a few hundred ms even at peak.

---

2. Estimation — the number that defines the design

> **Candidate:** Let me size the spike, because that dictates everything.

```
10M burgers, but how many PEOPLE tap? Way more than 10M — say 50M users
all pile in, most in the first ~60 seconds.

If 40M requests land in the first 60s:
  peak ≈ 40,000,000 / 60 ≈ 670,000 requests/sec   (and burstier at t=0)

Each request wants to touch the SAME counter. A single Postgres row doing
SELECT ... FOR UPDATE at 670k/s? It will melt — that row becomes a global lock
and every request serializes behind it. Throughput collapses to a few
thousand/sec and everyone else times out.

=> The whole problem is: how do we decrement one global counter 10M times,
   under 670k/s of contention, without a single hot lock and without overselling.
```

Candidate: So the core challenge is a **single hot key under enormous concurrent writes**. Everything I design bends toward removing that contention.

---

3. The naive V1 (and why it dies)

> **Candidate:** Let me state the obvious version first so we agree on why it fails.

```
       [ App ] --POST /claim--> [ Server ] --> [ Postgres ]
                                                BEGIN;
                                                SELECT remaining FROM promo FOR UPDATE;
                                                if remaining > 0:
                                                   UPDATE promo SET remaining = remaining-1;
                                                   INSERT claim(user_id) ...
                                                COMMIT;
```

Why it dies: `SELECT ... FOR UPDATE` on the single `promo` row means **every one of the 670k requests/sec serializes on one row lock**. Even if a transaction takes 1 ms, that's a ceiling of ~1,000 grants/sec. We'd need ~2.7 *hours* to hand out 10M, not 10 minutes — and the lock queue would time out long before that. **The single hot row is the enemy.**

---

4. The key idea — pre-mint the inventory, don't decrement a counter

> **Candidate:** The trick with fixed-inventory giveaways: **don't compute scarcity at request time with a lock — pre-create the 10M scarce things ahead of time and hand them out with an atomic "grab one" operation** that no single lock serializes.

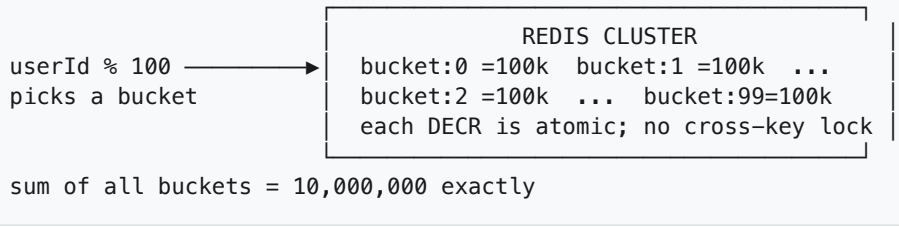
I'll use **Redis** as the front-line arbiter of scarcity, because it gives me atomic single-key operations at 100k+ ops/sec per shard, and I can **sharurt the inventory across many keys** to kill the hot-key problem.

Option A — atomic counter with `DECR` (simple, one hot key)

Redis `DECR` is atomic and single-threaded-safe. `remaining = DECR burgers`. If the result is `>= 0`, granted; if `< 0`, sold out. No oversell — Redis serializes DECRs internally at memory speed (~100k+/s per instance). But it's still **one hot key** on one shard → capped by that shard's throughput.

Option B — sharded counters (kills the hot key) ✅ my choice

Split the 10M into **N buckets**, e.g. 100 Redis counters of 100,000 each, spread across the cluster. A request hashes to a bucket (e.g. by `userId % 100`) and does `DECR bucket:{k}`. Now the 670k/s is spread across 100 keys on many shards → ~6,700/s per key, which Redis eats easily.



Edge case the interviewer will poke: what if *your* bucket hits 0 but others still have stock? A user could see "sold out" while burgers remain. Fix: on a local bucket miss, **try a few sibling buckets** (a bounded fan-out, e.g. probe 3 random buckets) before declaring sold out. Near the very end this "steals" from neighbors so we truly exhaust all 10M. The residual imbalance is tiny relative to 10M.

---

5. Exactly-once per user (the second correctness rule)

> **Interviewer:** Good. Now — one per user. How do you stop a user getting two?

Candidate: Two atomic guards, in order:

1. **Dedup set first, decrement second.** Before touching a burger bucket, claim the user: `SET claimed:{userId} 1 NX` (set-if-not-exists, atomic).
2. If it returns "already set" → the user already claimed → respond `ALREADY_CLAIMED` ("on the way"). We never touch inventory.
3. If it returns "set OK" → this is their first tap → proceed to grab a burger.
4. **Only then** `DECR bucket`. If the bucket (and probes) are all empty → `SOLD_OUT` — and we must **release the user guard** (`DEL claimed:{userId}`) so we didn't burn their one chance on a sold-out response. (Or mark them as "eligible but sold out.")

```
POST /claim  (Idempotency-Key = userId, so retries are free)
  │
  ├─ SET claimed:{userId} NX
  │     ├─ already exists ──────────────► return ALREADY_CLAIMED  (on the way)
  │     └─ newly set ↓
  ├─ DECR bucket:{userId % 100}
  │     ├─ result >= 0 ─────────────────► enqueue grant, return GRANTED(code)
  │     └─ result <  0 → probe 3 siblings
  │           ├─ found one ─────────────► GRANTED(code)
  │           └─ all empty ─────────────► DEL claimed:{userId}; return SOLD_OUT
```

Why `SET NX` and not a DB unique constraint? A `UNIQUE(user_id)` in Postgres would work for correctness, but it puts the per-user check back on the database under 670k/s. Redis `SET NX` is O(1) in memory and shards by user key naturally — no hot spot, since users spread across the keyspace.

---

6. Async — get the durable write off the hot path

> **Interviewer:** Redis is in memory. If it restarts, you've lost who got burgers. And legal wants an auditable record of every grant. How?

Candidate: Right — Redis is the **fast arbiter of scarcity**, but it is **not the source of truth for the audit log.** The moment a burger is granted, I **enqueue the grant** to a durable log and return to the user immediately. A pool of workers persists it asynchronously.

```
[claim granted in Redis]
      │  push {userId, code, ts, bucket}
      ▼
 [ Kafka topic: burger-grants ]  ── partitioned by userId, durable, replicated
      │
      ▼
 [ Consumer workers ] ──► [ Postgres: grants table ]  (source of truth / audit)
                      └─► [ Email/SMS: "your burger is on the way" ]
```

- **The user doesn't wait for Postgres or the email.** They get their code the instant Redis says yes. The DB write and notification happen off the critical path.
- **Kafka gives durability + replay.** If a consumer crashes, messages aren't lost; we replay from the last committed offset. Partitioning by `userId` keeps per-user ordering.
- **Reconciliation:** a batch job sums Redis grants vs. Postgres rows to confirm we granted exactly ≤10M. Redis is authoritative for the *count*; Postgres is authoritative for the *record*.

Why Kafka and not just SQS? We want **high throughput + ordered, replayable partitions** and a durable log we can re-process for audit/reconciliation. SQS is fine for simple fire-and-forget work queues, but Kafka's partitioned, retained log is the better fit when the event stream itself is the audit trail and we may need to replay 10M events.

---

7. Handling the spike at the edge — shed load before it hits Redis

> **Interviewer:** 670k/s is still a lot to even accept. What sits in front?

Candidate: Several layers *before* Redis, because the cheapest request to handle is the one you reject early:

```
[ Clients ]
     │   (jittered start: client adds random 0–5s delay to flatten t=0 spike)
     ▼
[ CDN / Edge ]  ── serves the static "claim" page; absorbs non-API traffic
     ▼
[ API Gateway + L7 Load Balancer ]
     │   - rate-limit per IP / per device (stops bots hammering)
     │   - admission control: if Redis is saturated, return SOLD_OUT/BUSY fast
     ▼
[ Stateless claim servers ]  (auto-scaled, hundreds of pods)
     ▼
[ Redis cluster: user guards + sharded burger buckets ]
     ▼
[ Kafka ] ──► [ workers ] ──► [ Postgres + notifications ]
```

Key load-shedding moves: - **Client-side jitter / virtual queue.** Instead of 40M taps at the same millisecond, the client (or a "waiting room" like a lightweight token issuer) spreads arrivals over the window. A **virtual waiting room** (à la ticketing sites) issues entry tokens at a controlled rate — the backend only ever sees a rate it can handle. - **Fast-path SOLD_OUT.** Once a global "sold out" flag flips (all buckets empty), the gateway can short-circuit and return SOLD_OUT from the edge without touching Redis at all. This is the graceful-degradation valve. - **Stateless servers** so we can horizontally scale to hundreds of pods; all state lives in Redis/Kafka.

---

8. Technology choices — what and why (the detailed part)

Concern	Choice	Why this, and why not the alternative
Scarcity arbiter (the counter)	**Redis (cluster), sharded counters**	Atomic `DECR` / `SET NX` at 100k+/s per shard, single-threaded so no oversell within a key; sharding across keys removes the hot spot. *Not Postgres row lock* — one hot row serializes all writers and caps us at ~1k/s.
Per-user dedup	**Redis `SET NX`** on `claimed:{userId}`	O(1) in memory, shards by user, no hot key. *Not a SQL `UNIQUE` constraint* — that pushes 670k/s of uniqueness checks onto the DB.
Source of truth / audit	**PostgreSQL** (`grants` table)	We need durable, queryable, auditable records with ACID; volume (10M rows) is trivial for Postgres. Written **async**, so its speed never gates the user.
Event pipeline	**Kafka**	Durable, partitioned, replayable log — doubles as the audit stream; ordered per `userId`. *Not SQS* — we want retention + replay for reconciliation, not just a work queue.

Concern	Choice	Why this, and why not the alternative
Notifications	**Async workers off Kafka**	"On the way" email/SMS is not on the critical path; decoupling means a slow email provider can't slow claims.
Edge	**CDN + API Gateway (L7)**	Static page from CDN; gateway does rate-limiting, admission control, and fast-path SOLD_OUT. L7 so we can route/limit on path + headers.
Waiting room	**Token-bucket virtual queue**	Converts an uncontrollable arrival spike into a controlled, backend-safe rate.

The one-sentence justification you'd say out loud: "*Redis is the fast, atomic, shardable front line that guarantees we never oversell and never double-grant; Postgres is the durable source of truth written asynchronously behind Kafka so its latency never touches the user; and a virtual waiting room plus edge rate-limiting flattens the spike so the backend only ever sees a rate it can serve.*"

---

9. Consistency model — what's strong, what's eventual

- **Strong / linearizable:** the burger count and the per-user guard. Redis single-key atomic ops give us this *per key*; sharding keeps correctness because the sum of buckets is a fixed 10M and each `DECR` is atomic. We never exceed 10M.
- **Eventual:** the Postgres audit record and the notification. A user has their code the instant Redis grants; the durable row and the SMS land a moment later. That's acceptable — the *promise* (you got a burger) is made atomically in Redis; the *paperwork* catches up.
- **The subtle risk:** a grant succeeds in Redis but the process crashes before enqueuing to Kafka → a burger is "spent" in the counter but has no record. Mitigation: write the grant event to Kafka **as part of** the grant path with at-least-once delivery, and reconcile; worst case we slightly *under*-count real grants (safe — never oversell), and reconciliation can top up unused inventory at the end.

---

10. Failure modes & graceful degradation

Failure	What happens	Mitigation
Redis shard dies	Its bucket's remaining burgers are stranded	Redis replicas (failover promotes a replica); sibling-bucket probing routes around a briefly-down shard; reconciliation recovers stranded count
Kafka lag / consumers slow	Audit rows + emails delayed	User is unaffected (already has code); consumers catch up; backpressure never reaches the claim path
Postgres down	Audit writes queue in Kafka	Kafka retains messages; workers drain when DB recovers — no data loss, no user impact
Traffic > capacity	Gateway sheds load	Rate-limit + waiting room + fast-path SOLD_OUT; users see a clean "busy, try again" not a crash

Failure	What happens	Mitigation
Bot / abuse	Fake claims	Per-device + per-IP limits at the gateway, `SET NX` per user, optional CAPTCHA in the waiting room

11. Follow-up curveballs

> **Interviewer:** What if a user claims but never picks up the burger — can we recycle it?

Candidate: Add a TTL/expiry on the grant. A sweeper job finds `GRANTED` records not redeemed within, say, 24h, and **returns them to inventory** by `INCR`-ing a bucket and clearing the user guard. This reopens a trickle of stock — handle it the same atomic way.

> **Interviewer:** Marketing wants a live "burgers remaining" counter on the homepage.

Candidate: That's a **read**, and it must NOT read the hot buckets 670k times/sec. Compute `sum(buckets)` on a timer (e.g. every 1–2s) in a background job, cache the total in a single read-only Redis key or push via the CDN, and let the homepage read *that*. Approximate, eventually-consistent, cheap — perfect for a display counter. Never let a cosmetic number contend with the real inventory keys.

> **Interviewer:** How do you test this before the real launch?

Candidate: Load-test with a synthetic 670k/s against a staging Redis cluster; run a **game-day** with the real waiting-room + gateway to validate admission control; and dry-run the reconciliation job to prove `sum(grants) ≤ 10M` holds under induced shard failures.

12. Deep-dive: "Why not pre-load 10M messages in Kafka/MQ and pull one per user?"

> **Interviewer:** Instead of counters, could I just put **10 million dummy messages** in a queue, and each user **pulls exactly one message** to claim a burger? Empty queue = sold out. Clean, no?

Candidate: It's a natural idea, and it *does* work for one variant — but not with Kafka, and for a pure counter it's the wrong tool. Let me separate the two cases.

Why Kafka specifically cannot do this

Kafka is a **distributed log, not a work queue.** Consumers don't "grab and remove" an individual message — a **consumer group** is assigned whole **partitions** and reads them **sequentially by offset**. There is no operation for "millions of independent users each atomically pop one distinct message and

mark it consumed." Kafka has no per-message claim/delete. So the "pull one message per user" model simply isn't how Kafka works.

```
Kafka reality:                              What the idea needs:
  partition 0: [m0 m1 m2 m3 ...]              millions of users each atomically
  partition 1: [m0 m1 m2 m3 ...]              pop exactly ONE arbitrary message
  -> consumers read SEQUENTIALLY,             and remove it -> Kafka has no such
     by offset, per assigned partition           per-message operation
```

A competing-consumer MQ (RabbitMQ / SQS) *can* — but it's worse here

RabbitMQ and SQS **do** support "pull one message, ack, it's gone" with competing consumers. So conceptually you could pre-load 10M messages and let each claim pop one; an empty queue means sold out. But for *this* problem it loses to a counter on every axis:

Concern	Queue of 10M messages	Sharded Redis `DECR`
Throughput at 670k/s	One queue caps at tens of thousands/s → you must split into many queues = **sharding again**, the same hot-spot you were avoiding	`DECR` is memory-speed; shard across keys and it's trivial
Per-user dedup	Queue does **not** stop a user claiming twice — two taps pop two messages. You **still need** `SET NX` per user	Same `SET NX` guard, unchanged
Crash semantics	Consumer pops a message, crashes before replying → message lost, or redelivered only after a visibility-timeout delay (extra latency + complexity)	Lost response just means the user retries with the same idempotency key; the `DECR` already happened at most once
Setup cost	Physically writing 10M messages into a broker before launch	Set 100 keys to 100,000 each
Latency per claim	ack / visibility-timeout / delete round-trip	one atomic op

The one case where the queue-of-tickets pattern *is* right

If each unit is a **distinct item**, not just a count — **10M unique coupon codes**, reserved seat IDs, serial numbers — then "pop one message" hands the user their *specific* code atomically, and a queue (or a Redis list with `LPOP`) is a clean fit. For a **pure count of identical burgers**, an atomic counter is simpler, faster, and cheaper.

> **Rule of thumb:** *Counter* when units are interchangeable (a count). *Queue / `LPOP` a list* when each unit is unique (a code/seat). Either way you still shard for throughput and still guard per-user with `SET NX`.

---

13. Deep-dive: the empty-local-bucket / stranded-stock problem

> **Interviewer:** Your sharded counters have a hole. What happens when a user's **local bucket is 0 but other buckets still have stock**? Walk me through it in detail.

Candidate: This is the real weakness of sharded counters, and it's worth being precise. First — **correctness is never at risk.** An atomic `DECR` never grants on a negative result, so we can *never oversell* the 10M cap no matter how skewed the buckets are. The problem is purely about the **tail**: fairness and giving away *all* 10M.

Why it happens

We route users by `userId % 100`. That hashing isn't perfectly uniform, and buckets drain at different rates. Near the end you get imbalance:

```
bucket:0  = 0     (drained)     user hashed here -> sees SOLD_OUT ...
bucket:1  = 0     (drained)
bucket:2  = 143   (still has stock!)  ... but 143 burgers sit here unclaimed
bucket:3  = 0
...
sum across buckets = 512 left, yet many users are told "sold out"
```

Two bad outcomes: **(a) unfairness** — a user sees "gone" while stock exists; **(b) under-giveaway** — we fail to hand out all 10M. Never over-giveaway.

Fix 1 — sibling probing (bucket stealing), done atomically

When the local bucket is empty, try a few other buckets before declaring sold out. Do it in a **Lua script** so the check-and-decrement is atomic (Redis runs the whole script single-threaded) — this avoids the racy "`DECR`, see -1, `INCR` it back" dance that leaves counters flickering negative:

```
-- KEYS = a handful of candidate bucket keys to try
for i = 1, #KEYS do
  local v = tonumber(redis.call('GET', KEYS[i]) or '0')
  if v > 0 then
    redis.call('DECR', KEYS[i])
    return i            -- granted from bucket i, atomically, no race
  end
end
return -1               -- all candidates truly empty
```

Bound the probe count (try ~3–5 buckets, not all 100). Otherwise, at the very end *every* user probes *every* bucket and you create a probing thundering-herd — trading one hot-key problem for an all-keys problem.

Fix 2 — track which buckets are still alive

Keep a Redis set `active_buckets`. When a bucket hits 0, `SREM` it from the set. A user with an empty local bucket does `SRANDMEMBER active_buckets` to pick a bucket that *definitely* has stock, instead of blind probing. A little bookkeeping for far fewer wasted probes.

Fix 3 — the hybrid two-phase design (recommended)

The cleanest answer, and the one I'd lead with:

```
PHASE 1 (bulk — ~99% of burgers, during the 670k/s storm):
  sharded counters, NO probing. Fully parallel, fast. Accept tiny imbalance.

PHASE 2 (tail — last small %, AFTER traffic has collapsed):
  a background aggregator sums buckets every ~1s. When total remaining is low,
  flip a flag -> route ALL remaining claims to ONE global counter holding the remainder.
```

The insight: the single-hot-key problem *only* existed because of the spike. By the time you're down to the last few thousand burgers, **most demand is already satisfied and traffic has collapsed** — so one global counter can drain the tail *exactly*, giving away all 10M with zero stranded stock. You get sharded throughput during the storm and single-counter exactness at the end.

Fix 4 — power-of-two-choices (prevents imbalance up front)

On each claim, pick **two** random buckets and decrement whichever has more stock. This simple trick keeps bucket levels remarkably even, so the empty-while-others-full case barely arises. Cheap insurance that attacks the cause rather than patching the symptom.

Fix 5 — background rebalancing

A job periodically sums buckets and redistributes remaining stock evenly. It works, but adds moving parts and its own races — I'd reach for the hybrid (#3) or power-of-two (#4) first.

> **What to say out loud:** "*The atomic `DECR` guarantees we never oversell regardless of skew — the cap is safe. The empty-bucket case only threatens fairness and giving away the full 10M. I'd combine power-of-two-choices to keep buckets balanced during the storm with a single-counter tail phase once traffic drops, so the last burgers drain exactly. Sibling probing via a bounded Lua script is the simpler fallback if I want one mechanism.*"

Summary — the mental model

The 10M burger giveaway is **not a scale-the-reads problem; it's a "decrement one global number 10M times under 670k/s without a hot lock and without overselling" problem.** The winning moves: **pre-shard the scarce inventory across many atomic Redis counters**, guard each user with an atomic `SET NX`, make the durable write and notification **async** behind Kafka, and **flatten the arrival spike** with a virtual waiting room + edge rate-limiting. Redis guarantees correctness at memory speed; Postgres is the async source of truth; the waiting room guarantees the backend only ever sees a rate it can serve.